BÁO CÁO THẨM ĐỊNH VÀ ĐÁNH GIÁ CHUYÊN MÔN - ĐỀ TÀI QUẢN LÝ GIẢI BÓNG ĐÁ CHUYÊN NGHIỆP

1. Đánh giá Tổng quan và Nhận diện Các Đặc trưng của Trí tuệ Nhân tạo (AI-Generated Hallmarks)

Báo cáo chuyên đề "Xây dựng hệ cơ sở dữ liệu quản lý một giải bóng đá chuyên nghiệp"[1] trình bày một quy trình phát triển cơ sở dữ liệu từ giai đoạn khảo sát nghiệp vụ, thiết kế Lược đồ Thực thể - Liên kết (ERD), chuyển đổi mô hình quan hệ, chuẩn hóa Boyce-Codd (BCNF), cho đến cài đặt phân mảnh phân tán. Đánh giá trên bề mặt, văn bản sở hữu một cấu trúc hoàn hảo, ngôn từ mượt mà và cách trình bày rành mạch. Tuy nhiên, khi đi sâu vào phân tích ngữ nghĩa, cấu trúc logic và bản chất toán học của các khối dữ liệu, báo cáo bộc lộ rõ những lỗ hổng học thuật nghiêm trọng. Dưới góc độ chuyên môn của một chuyên gia về Hệ quản trị Cơ sở dữ liệu (RDBMS), các bằng chứng cho thấy tài liệu này mang đậm các đặc trưng điển hình của một văn bản do Trí tuệ Nhân tạo (Large Language Models) tự động sinh ra, thiếu đi sự tư duy phản biện và độ chính xác thực chứng của một nghiên cứu sinh con người.

1.1. Ảo giác Dữ liệu (Data Hallucination) và Sự Sụp đổ của Logic Toán học

Minh chứng không thể chối cãi nhất về việc can thiệp của AI tạo sinh nằm ở việc bịa đặt số liệu thực nghiệm. Trí tuệ nhân tạo thường xuất sắc trong việc mô phỏng cú pháp ngôn ngữ nhưng lại liên tục thất bại trước các ràng buộc toán học tất định (deterministic constraints) nếu không được lập trình công cụ giải toán riêng biệt. Điều này thể hiện rõ ràng qua "Bảng 3.1: Kết quả truy vấn bảng xếp hạng mùa giải"[1].

Báo cáo thiết lập tham số đầu vào của bài toán là một giải đấu gồm $N = 6$ đội bóng, thi đấu theo thể thức vòng tròn khép kín hai lượt (sân nhà và sân khách). Cấu trúc của một đồ thị vòng tròn có hướng (directed round-robin graph) quy định tổng số trận đấu của toàn mùa giải phải tuân theo công thức tổ hợp chặt chẽ. Cụ thể, mỗi đội sẽ thi đấu với 5 đội còn lại, nhân với 2 lượt đi và về, chia đôi để loại bỏ sự lặp lại của các cặp đấu, cho ra chính xác 30 trận đấu toàn giải.

Chỉ số Thống kê	Dữ liệu Báo cáo Trình bày	Phân tích Toán học Thực tế	Kết luận Khớp nối
Tổng số trận Thắng (W)	$8 + 6 + 5 + 4 + 2$ trận	Mỗi trận đấu có phân định thắng bại sinh ra 1 trận Thắng và 1 trận Thua. Do đó $\sum W$ phải	**Mâu thuẫn** ($25 \neq 23$)

		luôn luôn bằng $\sum L$.	
Tổng số trận Thua (L)	$0+2+2+3+7$ trận	Tổng Thua phải bằng tổng Thắng trên toàn bộ hệ thống đồ thị giải đấu.	**Mâu thuẫn** ($23 \neq 25$)
Số trận Hòa (D)	$2+2+3+3+1$ lượt	Mỗi trận Hòa được cộng cho 2 đội, số trận thực tế là $12/2=6$ trận.	**Hợp lệ về mặt số chẵn**
Tổng số trận đấu (Matches)	Tối thiểu $25(\text{thắng})+6(\text{hò}$ trận	Với $N=6$, đấu 2 lượt, tổng số trận đấu theo công thức toán học là chính xác 30 trận.	**Vượt quá giới hạn** ($31>30$)
Hiệu số Bàn thắng/Thua	Tổng bàn thắng (GF) = 84, Tổng bàn thua (GA) = 84	Trong hệ thống khép kín, một bàn thắng của đội này là bàn thua của đội kia. $\sum GF \equiv \sum GA$.	**Hợp lệ** ($84=84$)

Sự chênh lệch giữa tổng số trận thắng (25) và tổng số trận thua (23) là một sai lệch tuyệt đối trong lý thuyết trò chơi có tổng bằng không (zero-sum game elements) của môn bóng đá[2]. Hơn nữa, việc số trận đấu bị đội lên thành 31 trận thay vì 30 trận như thiết kế[1] chứng minh một cách không thể chối cãi rằng: Sinh viên hoàn toàn không thực thi hệ thống cơ sở dữ liệu để chạy câu lệnh SELECT đếm bảng xếp hạng. Số liệu này là sự chắp vá ngẫu nhiên của mô hình ngôn ngữ lớn nhằm tạo ra một bảng dữ liệu trông có vẻ hợp lý về mặt thị giác nhưng trống rỗng về mặt logic toán học. Sự thiếu trung thực trong việc báo cáo số liệu thực nghiệm là một vi phạm nghiêm trọng về tính liêm chính học thuật.

1.2. Sự Rập Khuôn Trong Văn Phong Phân Tích Cơ Sở Dữ Liệu

Ngoài lỗi về toán học, văn bản bộc lộ tư duy "khớp mẫu" (pattern matching) của AI trong việc chứng minh Dạng chuẩn Boyce-Codd (BCNF). Ở toàn bộ Chương 2, người viết lặp lại nguyên vẹn một công thức phân tích cơ học cho 11 bảng dữ liệu: xác định tập thuộc tính, xác định tập phụ thuộc hàm tối thiểu, xác định khóa ứng viên, và kết luận "Vế trái là siêu khóa"[1].
Một quy trình chuẩn hóa CSDL chuyên sâu đòi hỏi sự biện luận về bao đóng của tập thuộc tính (closure of attributes, X^+) để chứng minh một tập thuộc tính thực sự là siêu khóa. Hơn thế nữa, BCNF chỉ thực sự khác biệt và có ý nghĩa so với Dạng chuẩn 3 (3NF) trong trường hợp lược đồ quan hệ sở hữu các khóa ứng viên gối lên nhau (overlapping candidate keys)[3]. Việc AI sinh ra các kết luận BCNF một cách lặp đi lặp lại cho các bảng danh mục đơn giản như SAN_VAN_DONG hay CAU_THU (vốn dĩ không có khóa gối nhau) cho thấy hệ thống không có khả năng tư duy khái niệm (conceptual reasoning) mà chỉ đang điền từ vào một biểu mẫu có sẵn. Việc nhầm lẫn giữa kiểu dữ liệu DATETIME (đặc trưng của MySQL) ở phần thiết kế Từ điển dữ liệu và TIMESTAMP (đặc trưng của PostgreSQL) ở phần cài đặt PL/pgSQL[1] càng củng cố thêm luận điểm này.

2. Phản Biện Kiến Trúc Dữ Liệu và Lược Đồ Thực Thể - Liên Kết (ERD)

Thiết kế cơ sở dữ liệu quan hệ yêu cầu việc xác định đúng các phụ thuộc nghiệp vụ để loại bỏ sự dư thừa, đồng thời duy trì khả năng mở rộng trong tương lai. Lược đồ được trình bày trong báo cáo chứa những lỗ hổng kiến trúc nghiêm trọng, vi phạm nguyên tắc "Nguồn chân lý duy nhất" (Single Source of Truth) và mô hình hóa dữ liệu thời gian (Temporal Data Modeling).

2.1. Dị Thường Dữ Liệu Dẫn Xuất (Derived Data Anomalies) Trong Bảng TRAN_DAU

Bảng TRAN_DAU được thiết kế lưu trữ các thuộc tính như BanThangNha, BanThangKhach, TheVangNha, TheVangKhach, TheDoNha, và TheDoKhach[1]. Đồng thời, hệ thống lại duy trì một bảng SU_KIEN_TRAN_DAU để ghi nhận chi tiết từng sự kiện ghi bàn và phạt thẻ theo từng phút[1]. Việc lưu trữ các trường dữ liệu tổng hợp ngay bên trong bảng giao dịch cốt lõi là một phản mẫu (anti-pattern) kinh điển. Các thuộc tính này bản chất là dữ liệu dẫn xuất, có thể dễ dàng suy ra từ việc thực hiện phép toán tập hợp COUNT() kết hợp GROUP BY trên bảng SU_KIEN_TRAN_DAU. Việc duy trì dữ liệu dẫn xuất ở dạng tĩnh tạo ra nguy cơ bất đồng bộ dữ liệu (Data Inconsistency) rất lớn. Báo cáo cố gắng xử lý vấn đề này bằng cách thiết lập một Trigger kiểm tra tính khớp nối (Listing 3.6)[1]. Tuy nhiên, giải pháp này sinh ra một điểm thắt nút (bottleneck) về hiệu năng.
Trong môi trường thực tế, bất kỳ sự điều chỉnh nào (ví dụ: trọng tài VAR hủy một bàn thắng sau trận đấu) sẽ yêu cầu việc cập nhật đồng thời ở cả hai bảng. Thiết kế tối ưu cho bài toán này là loại bỏ hoàn toàn các cột tổng hợp khỏi bảng TRAN_DAU và sử dụng một Khung nhìn (View) hoặc Khung nhìn cụ thể hóa (Materialized View) để tính toán tỉ số theo thời gian thực. Điều này đảm bảo tính tuân thủ tuyệt đối chuẩn hóa (thường là 3NF/BCNF) và bảo vệ hệ thống khỏi các dị thường cập nhật.

2.2. Sự Thiếu Sót Của Khung Thời Gian (Temporal Framework) Trong

Hợp Đồng

Mô hình hóa nghiệp vụ của bảng HOP_DONG_CAU_THU chứa hai khóa ngoại là MaDoiChuQuan và MaDoiHienTai[1]. Giải pháp này được đưa ra để giải quyết bài toán "Cho mượn cầu thủ". Dưới lăng kính chuyên môn, đây là một cách tiếp cận cực kỳ ngây ngô và thiếu tính lịch sử.
Khi thực thể thay đổi trạng thái theo thời gian, cơ sở dữ liệu phải áp dụng các kỹ thuật thiết kế Kích thước thay đổi chậm (Slowly Changing Dimensions - SCD), đặc biệt là SCD Loại 2, hoặc sử dụng các Bảng thời gian (Temporal Tables) để lưu vết[6].

- **Mất mát dữ liệu lịch sử:** Nếu cầu thủ A được đội X cho đội Y mượn từ tháng 1 đến tháng 6, thuộc tính MaDoiHienTai sẽ là Y. Khi cầu thủ quay lại đội X vào tháng 7, thuộc tính này bị ghi đè thành X. Hệ thống hoàn toàn bốc hơi thông tin rằng cầu thủ A từng thi đấu cho Y.
- **Sai lệch bản thể luận (Ontology):** Hợp đồng chuyên nghiệp (Professional Contract) và Thỏa thuận cho mượn (Loan Agreement) là hai thực thể pháp lý hoàn toàn khác biệt. Hợp đồng chuyên nghiệp tồn tại giữa 2 bên (Câu lạc bộ và Cầu thủ), trong khi Thỏa thuận cho mượn tồn tại giữa 3 bên (Câu lạc bộ chủ quản, Câu lạc bộ mượn, Cầu thủ) và có các điều khoản chia sẻ cấu trúc lương phức tạp. Việc nhồi nhét hai khái niệm này vào cùng một bảng và phân biệt bằng cờ TrangThai dẫn đến việc thuộc tính MucLuong trở nên vô nghĩa khi không xác định rõ dòng tiền thuộc về tổ chức nào.

Một kiến trúc sư CSDL chuyên nghiệp sẽ phân rã bảng này thành HOP_DONG_CHINH_THUC và LICH_SU_CHO_MUON, liên kết với nhau bằng các khóa ngoại mạnh. Để ngăn chặn các hợp đồng bị chồng chéo về mặt thời gian, thiết kế phải sử dụng đặc tả ràng buộc loại trừ của PostgreSQL, ví dụ: EXCLUDE USING gist (MaCauThu WITH =, daterange(NgayBatDau, NgayKetThuc) WITH &&)[6]. Sự thiếu vắng các kiến trúc này minh chứng cho sự yếu kém trong kỹ năng phân tích thiết kế hệ thống.

3. Phân Tích Sự Ngụy Biện Trong Chuẩn Hóa Boyce-Codd (BCNF)

Chương 2 của báo cáo đề cập đến việc chuẩn hóa BCNF cho 11 lược đồ quan hệ. Sự trình bày ở đây không chỉ hời hợt mà còn cho thấy sự nhầm lẫn căn bản về lý thuyết cơ sở dữ liệu quan hệ, đi ngược lại với các định nghĩa hình thức (formal definitions) của Zaniolo và Codd[4].

3.1. Sự Khác Biệt Cốt Lõi Giữa 3NF Và BCNF Bị Bỏ Qua

Báo cáo trình bày việc kiểm tra BCNF theo phương pháp: xác định khóa chính, liệt kê các thuộc tính phụ thuộc vào khóa chính, và kết luận về trái là siêu khóa[1]. Cách chứng minh này thực chất chỉ dừng lại ở Dạng chuẩn 2 (2NF) hoặc Dạng chuẩn 3 (3NF).
Lý thuyết cơ sở dữ liệu chỉ ra rằng: Định nghĩa của 3NF cho phép tồn tại một phụ thuộc hàm $X \to A$ trong đó X không phải là siêu khóa, miễn là A là một thuộc tính khóa (prime attribute - thuộc tính cấu thành nên một khóa ứng viên nào đó)[3]. BCNF là một dạng chuẩn chặt chẽ hơn, triệt tiêu ngoại lệ này, đòi hỏi *mọi* định thức (determinant) đều bắt buộc phải là siêu khóa[3]. Do đó, BCNF chỉ phát sinh sự khác biệt thực sự so với 3NF khi và chỉ khi lược đồ quan hệ có **nhiều khóa ứng viên gối lên nhau (multiple overlapping candidate keys)**[3]. Việc nhóm tác giả không hề đề cập đến khái niệm khóa ứng viên gối nhau, không chỉ ra được bất kỳ thuộc tính

khóa nào bị phụ thuộc ngược, chứng tỏ quy trình chứng minh BCNF trong báo cáo hoàn toàn chỉ là hình thức, không đi vào bản chất của lý thuyết chuẩn hóa.

3.2. Sự Phá Vỡ BCNF Do Các Phụ Thuộc Hàm Ngầm Định (Implicit Functional Dependencies)

Phân tích sâu hơn vào cấu trúc dữ liệu, báo cáo đã bỏ sót hàng loạt các phụ thuộc hàm phát sinh từ quy tắc nghiệp vụ, làm phá vỡ toàn bộ cấu trúc chuẩn hóa đã tuyên bố.

Xem xét bảng HOP_DONG_CAU_THU(MaHD, MaCauThu, MaDoiChuQuan, MaDoiHienTai, SoAo, NgayBatDau, NgayKetThuc, MucLuong)[1]. Báo cáo khẳng định chỉ có duy nhất một khóa ứng viên là $K = \{MaHD\}$ và đạt chuẩn BCNF[1]. Tuy nhiên, hãy đối chiếu với hai quy tắc nghiệp vụ cốt lõi ở Mục 1.1.2:

1. "Tại mỗi thời điểm, mỗi cầu thủ chỉ được thi đấu cho đúng một đội bóng."[1]
2. "Trong cùng một đội bóng, các cầu thủ phải có số áo khác nhau."[1]

Từ đây, ta có thể xây dựng các phụ thuộc hàm theo thời gian (Temporal Functional Dependencies) cực kỳ quan trọng:

- FD_1 : $\{MaCauThu, NgayBatDau, NgayKetThuc\} \rightarrow \{MaDoiHienTai, SoAo, \dots\}$
- FD_2 : $\{MaDoiHienTai, SoAo, NgayBatDau, NgayKetThuc\} \rightarrow \{MaCauThu\}$

Sự xuất hiện của FD_1 và FD_2 xác lập thêm hai khóa ứng viên mới cho quan hệ này, đó là $K_2 = \{MaCauThu, NgayBatDau, NgayKetThuc\}$ và $K_3 = \{MaDoiHienTai, SoAo, NgayBatDau, NgayKetThuc\}$. Báo cáo hoàn toàn không nhìn thấy sự tồn tại của đa khóa ứng viên này, dẫn đến một sự đánh giá sai lầm về trạng thái chuẩn hóa. Việc không nhận diện được K_3 khiến cấu trúc hiện tại dễ bị chèn dữ liệu dị thường, trong đó hai cầu thủ khác nhau có thể vô tình mang cùng một số áo tại cùng một thời điểm cho cùng một câu lạc bộ nếu không có sự can thiệp của mã nguồn ứng dụng (application code).

Tương tự đối với lược đồ PHAN_CONG_TRONG_TAI(MaTran, VaiTro, MaTrongTai)[1]. Báo cáo khẳng định hệ thống đạt BCNF vì $\{MaTran, VaiTro\} \rightarrow MaTrongTai$ và $\{MaTran, MaTrongTai\} \rightarrow VaiTro$ [1]. Cả hai định thức đều là siêu khóa. Tuy nhiên, quy tắc nghiệp vụ định rõ: "Trong cùng một vòng đấu, một trọng tài chỉ điều khiển tối đa một trận đấu"[1]. Điều này sinh ra một phụ thuộc hàm ngoại lai (Inter-relational functional dependency): $\{MaTrongTai, VongDau\} \rightarrow MaTran$. Thuộc tính VongDau không tồn tại trong bảng PHAN_CONG_TRONG_TAI, mà phải được chiếu từ bảng TRAN_DAU. Lược đồ này vì vậy vi phạm các điều kiện về phụ thuộc đa trị (Multivalued dependencies) và sẽ không thể duy trì tính toàn vẹn nếu chỉ xét chuẩn BCNF đơn tuyến. Nó yêu cầu một sự chuẩn hóa tiến tới Dạng chuẩn 4 (4NF)[9] hoặc phải triển khai các ràng buộc kiểm tra cấp độ Assertion,

thay vì ngụy biện bằng Dạng chuẩn Boyce-Codd.

4. Sai Phạm Nghiêm Trọng Về Điều Khiển Đồng Thời (Concurrency) Và Ngôn Ngữ PL/pgSQL

Điểm yếu chí mạng nhất của báo cáo tập trung ở Chương 3, nơi trình bày việc cài đặt các hàm kích hoạt (Triggers) và thủ tục lưu trữ (Stored Procedures). Nhóm tác giả tiếp cận việc viết mã cho hệ quản trị CSDL theo tư duy của một luồng xử lý đơn tuyến (Single-threaded), hoàn toàn phớt lờ các lý thuyết phức tạp về Kiểm soát đồng thời đa phiên bản (Multi-Version Concurrency Control - MVCC) của PostgreSQL.

4.1. Sự Tàn Phá Của "Điều Kiện Tranh Chấp" (Race Conditions) Trong Việc Đếm Bằng Trigger

Cơ chế cô lập giao dịch (Transaction Isolation Level) mặc định của PostgreSQL là READ COMMITTED[10]. Ở chế độ này, một truy vấn đọc (SELECT) chỉ nhìn thấy dữ liệu từ các giao dịch đã được xác nhận (committed) trước thời điểm câu lệnh đó bắt đầu thực thi[10]. Các thay đổi từ các giao dịch đang diễn ra (in-flight transactions) hoàn toàn bị che giấu. Điều này dẫn đến các hiện tượng bóng ma (Phantom Reads) hoặc mất mát cập nhật (Lost Updates) khi xử lý logic Đọc-Sửa-Ghi (Read-Modify-Write)[10].

Quan sát Listing 3.5[1], hàm Trigger kích hoạt để kiểm tra số lượng cầu thủ tối đa được đăng ký mỗi trận:

```
SQL
SELECT COUNT * INTO v_so_cau_thu FROM DANG_KY_THI_DAU
WHERE MaTran = NEW.MaTran AND MaDoi = NEW.MaDoi
IF v_so_cau_thu >= 18 THEN RAISE EXCEPTION ...
```

Bản thiết kế này tiềm ẩn một lỗi Race Condition cực kỳ nguy hiểm có thể phá vỡ tính đúng đắn của dữ liệu ngay lập tức dưới áp lực của môi trường thực tế[10]. Hãy xem xét kịch bản sau: Đội bóng đã đăng ký 17 cầu thủ. Hai quản lý đội bóng đồng thời mở ứng dụng và gửi yêu cầu đăng ký cho cầu thủ thứ 18 và 19 vào cùng một phần nghìn giây.

- **Thời điểm T_1 :** Giao dịch A của Quản lý 1 thực thi lệnh SELECT COUNT(*), đếm được 17 cầu thủ.
- **Thời điểm T_2 :** Giao dịch B của Quản lý 2 thực thi lệnh SELECT COUNT(*). Do Giao dịch A chưa commit, Giao dịch B vẫn chỉ nhìn thấy 17 cầu thủ (Phantom Read)[10].
- **Thời điểm T_3 :** Giao dịch A kiểm tra $17 < 18$, điều kiện thỏa mãn, tiến hành INSERT bản ghi cầu thủ thứ 18 và COMMIT.
- **Thời điểm T_4 :** Giao dịch B kiểm tra $17 < 18$, điều kiện vẫn thỏa mãn, tiến hành

INSERT bản ghi cầu thủ thứ 19 và COMMIT.

Hệ quả là danh sách thi đấu của đội bóng có 19 cầu thủ, vi phạm hoàn toàn giới hạn 18 người. Lỗi vi phạm tính toàn vẹn này xảy ra thầm lặng mà không sinh ra bất kỳ lỗi hệ thống nào. Vấn đề tương tự xuất hiện tại Trigger kiểm tra 30 cầu thủ của câu lạc bộ (Listing 3.3) và Trigger kiểm tra số lượng trận đấu của trọng tài (Listing 3.4)[1].

Phương pháp giải quyết Concurrency	Cơ chế hoạt động	Nhược điểm / Ưu điểm
SELECT ... FOR UPDATE	Yêu cầu khóa hàng độc quyền (Exclusive Row Lock). Giao dịch B phải chờ Giao dịch A hoàn tất[13].	Chậm trong môi trường có tính tranh chấp cao, nguy cơ Deadlock nếu không khóa theo thứ tự nhất định[13]. Ưu điểm: Đảm bảo độ chính xác tuyệt đối ở cấp độ hàng.
Advisory Locks	Khóa cấp ứng dụng do người dùng định nghĩa, sử dụng hàm pg_advisory_xact_lock()[15].	Linh hoạt, tránh khóa các hàng không cần thiết. Đòi hỏi quản lý khóa phức tạp từ tầng ứng dụng.
SERIALIZABLE Isolation	Mức cô lập cao nhất. CSDL tự động phát hiện chu trình phụ thuộc và hủy giao dịch có nguy cơ gây sai lệch (Serialization Failure)[10].	Giảm mạnh hiệu năng do phải theo dõi biểu đồ khóa. Đòi hỏi ứng dụng phải tự động thử lại (retry logic) khi gặp lỗi 40001[12].

Để sửa chữa mã nguồn trong báo cáo, sinh viên bắt buộc phải sử dụng cơ chế cấp phát khóa độc quyền (Pessimistic Locking) bằng cách bổ sung lệnh SELECT ... FOR UPDATE[10]. Thay vì chỉ thực thi truy vấn COUNT vô giá trị, giao dịch phải khóa bản ghi liên quan tại bảng TRAN_DAU để buộc các luồng chèn dữ liệu phía sau phải xếp hàng chờ (Serialize the execution flow). Ví dụ sửa lỗi:

```
SQL
PERFORM 1 FROM TRAN_DAU WHERE MaTran = NEW.MaTran FOR NO KEY UPDATE;
SELECT COUNT * INTO v_so_cau_thu FROM DANG_KY_THI_DAU ...
```

Sự vắng mặt của việc xử lý kiểm soát đồng thời (Concurrency Control) khẳng định mã nguồn

chỉ là sản phẩm tạo sinh thô sơ và không có giá trị áp dụng thực tiễn.

4.2. Sự Khác Biệt Giữa Thủ Tục (Procedure) Và Hàm (Function)

Báo cáo trình bày mục 3.3: "Cài đặt năm thủ tục giao dịch thực thi nghiệp vụ"[1]. Mã nguồn tại Listing 3.8 định nghĩa CREATE OR REPLACE PROCEDURE sp_phan_cong_to_trong_tai(...)[1].

Kể từ phiên bản 11, PostgreSQL giới thiệu khả năng tạo Thủ tục lưu trữ (Stored Procedures) bằng lệnh CREATE PROCEDURE, phân biệt rạch ròi với CREATE FUNCTION. Lý do kiến trúc duy nhất để sử dụng Procedure thay cho Function là khả năng thực thi các câu lệnh kiểm soát giao dịch (Transaction Control) như COMMIT và ROLLBACK ngay bên trong khối mã[23]. Các hàm (FUNCTION) trong PostgreSQL không có quyền bắt đầu hay kết thúc các giao dịch top-level, chúng chỉ kế thừa ngữ cảnh giao dịch của khối lệnh gọi chúng[23].

Khi kiểm tra mã nguồn Listing 3.8 và toàn bộ các Listing khác trong phần 3.3[1], người đánh giá không hề thấy bất kỳ lệnh COMMIT hay ROLLBACK nào được sử dụng bên trong khối lệnh BEGIN...END. Mã nguồn sử dụng PROCEDURE nhưng cấu trúc hoàn toàn là của FUNCTION[28]. Việc gọi CALL hay SELECT trở nên vô nghĩa khi không tận dụng được lợi thế của giao dịch. Điều này chứng tỏ sinh viên hoàn toàn thiếu hụt kiến thức về nguyên lý thiết kế CSDL sâu thẳm, chỉ chép mã theo cấu trúc bề ngoài mà bỏ qua ngữ nghĩa kỹ thuật.

4.3. Hiểm Họa Từ Bảng Đột Biến (Mutating Table) Và Cấu Trúc Trigger

Trong Trigger 3.2.5 trg_check_khop_ban_thang_va_the, sinh viên sử dụng biến cố AFTER INSERT OR UPDATE ON TRAN_DAU để truy vấn ngược lại vào bảng SU_KIEN_TRAN_DAU và đếm số lượng bàn thắng, sau đó kiểm tra chéo với bản ghi của TRAN_DAU[1].

Mặc dù PostgreSQL xử lý lỗi Bảng Đột Biến (Mutating Table) tốt hơn Oracle (trong Oracle sẽ ném ra lỗi ORA-04091: table is mutating nếu một trigger mức hàng truy vấn chính bảng đang thay đổi[29]), nhưng việc ràng buộc cứng một ngoại lệ (RAISE EXCEPTION) khi tổng số không khớp tạo ra một thiết kế thảm họa về mặt chu trình sống của dữ liệu. Nếu trọng tài nhập sai một thẻ vàng vào hệ thống sự kiện, và sau đó cố gắng kết thúc trận đấu, Trigger sẽ phát hiện sự không khớp và từ chối cập nhật trạng thái trận đấu. Dữ liệu bị kẹt ở trạng thái "Chưa Kết Thúc" mãi mãi. Trong thế giới thực, các ràng buộc tổng hợp đa bảng như thế này nên được triển khai qua các tác vụ nền (Batch processing/Data auditing) hoặc phát ra cảnh báo (Soft warning/Log table) thay vì sử dụng Hard-block để hủy bỏ toàn bộ giao dịch, gây sụp đổ luồng tác nghiệp của người dùng[32].

4.4. Tính Toán Thời Gian Phá Hủy Chỉ Mục (Index Suppression)

Tại Listing 3.7 (sp_tao_lich_thi_dau), quy tắc kinh doanh yêu cầu các trận đấu trên cùng một sân vận động phải cách nhau tối thiểu 4 giờ đồng hồ. Sinh viên xử lý bằng truy vấn:

```
SQL
ABS EXTRACT EPOCH FROM (NgayGio - p_ngay_gio)) < 14400
```

Cú pháp này hợp lệ về mặt ngữ pháp PostgreSQL (trừ hai giá trị thời gian trả về một INTERVAL,

sau đó trích xuất thành giây)[1]. Tuy nhiên, việc áp dụng các hàm ABS() và EXTRACT() trực tiếp lên cột dữ liệu NgayGio khiến bộ tối ưu hóa truy vấn (Query Optimizer) của PostgreSQL bị "mù". Nó không thể sử dụng bất kỳ chỉ mục B-Tree nào trên cột NgayGio, dẫn đến việc cơ sở dữ liệu phải thực hiện Quét toàn bộ bảng (Full Table Scan) cho mọi giao dịch lên lịch thi đấu, làm suy thoái hiệu năng tuyến tính (Linear degradation).
Thay vì tính toán toán học thủ công (Manual Time Arithmetic), các kỹ sư CSDL giàu kinh nghiệm sẽ cấu hình PostgreSQL tự động xử lý thông qua Kiểu Dữ liệu Khoảng (Range Types) và Ràng buộc Loại trừ (Exclusion Constraints)[6]. Cú pháp chuẩn học thuật để bảo vệ sự vô hướng thời gian:

```
SQL
ALTER TABLE TRAN_DAU ADD CONSTRAINT chong_chong_cheo_thoi_gian
EXCLUDE USING gist (
    MaSan WITH =
    tsrange(NgayGio, NgayGio + interval '4 hours') WITH &&
)
```

Giải pháp này sử dụng cấu trúc cây GiST index để đối chiếu thời gian bị gối lên nhau &&, hoạt động ở độ phức tạp $O(\log n)$ và bảo mật dữ liệu tuyệt đối ở cấp độ vật lý của cơ sở dữ liệu mà không cần phải viết một dòng mã PL/pgSQL đầy rủi ro.

5. Đánh Giá Thiết Kế Cơ Sở Dữ Liệu Phân Tán (Distributed Database)

Chương 4 cung cấp nội dung về Cơ sở dữ liệu phân tán. Sinh viên tiếp tục phơi bày sự nhầm lẫn giữa khái niệm phân mảnh không gian mạng (Network Distribution) và phân vùng logic nội bộ (Logical Partitioning)[1].

5.1. Mô Phỏng Phân Tán "Giả Trân" Bằng Lược Đồ (Schema)

Theo báo cáo: *"Mô phỏng ba trạm vật lý thông qua ba không gian tên độc lập trên cùng một máy chủ"* và chạy các lệnh CREATE SCHEMA site1_mienbac; CREATE TABLE site1_mienbac.TRAN_DAU_1 AS SELECT...[1].
Hệ thống phân tán thực sự (Distributed Database System - DDBS) xoay quanh các bài toán cốt lõi về Tính sẵn sàng (Availability), Phân vùng mạng (Partition Tolerance theo định lý CAP), và Giao thức Xác nhận hai giai đoạn (Two-Phase Commit - 2PC). Việc chỉ sử dụng CREATE SCHEMA để tách biệt các bảng hoàn toàn là sự chia tách logic (Logical Separation). Việc di chuyển dữ liệu bằng lệnh INSERT INTO ... SELECT chỉ tạo ra các bản sao tĩnh (Static snapshots). Nếu dữ liệu ở miền Bắc thay đổi, cơ sở dữ liệu tại Trung tâm hoàn toàn không nhận được bản cập nhật vì sinh viên không hề cấu hình bất kỳ trình bao bọc dữ liệu ngoại lai nào như postgres_fdw (Foreign Data Wrapper) hoặc các công cụ sao chép logic (Logical Replication /

Pub-Sub)[7].

5.2. Các Vị Từ Phân Mảnh Tĩnh Trái Quy Tắc (Hardcoded Minterm Predicates)

Khi thực hiện phân mảnh ngang (Horizontal Fragmentation) cho bảng SAN_VAN_DONG, báo cáo đưa ra các vị từ:

$$p_1 : MaSan \in \{\text{'SAN_HD'}, \text{'SAN_MD'}, \text{'SAN_LT'}, \text{'SAN_TT'}\}$$ [1].

Theo lý thuyết CSDL phân tán, một tập các vị từ phân mảnh phải đảm bảo ba quy tắc: Tính đầy đủ (Completeness), Tính tái thiết (Reconstruction), và Tính phân tách (Disjointness). Vị từ p_1 vi phạm nghiêm trọng tính Đầy đủ. Việc "mã hóa cứng" (hardcode) các ID sân vận động vào điều kiện phân mảnh đồng nghĩa với việc: Nếu Liên đoàn bóng đá cấp phép cho một sân vận động mới ở miền Bắc (ví dụ SAN_CP), cơ sở dữ liệu sẽ sụp đổ. Bản ghi mới không thỏa mãn cả p_1 lẫn p_2, khiến bộ định tuyến (router) không biết phân phối dữ liệu về trạm nào.

Thiết kế chính xác yêu cầu lược đồ SAN_VAN_DONG phải bổ sung một thuộc tính phân loại (Categorical attribute) như KhuVuc VARCHAR(20). Khi đó, các vị từ sẽ trở nên tổng quát và bao trùm mọi khả năng: $p_1 : KhuVuc = \text{'MienBac'}$.

5.3. Gánh Nặng Từ Chiến Lược Nhân Bản Toàn Phần (Full Replication)

Báo cáo đề xuất nhân bản toàn phần các bảng DOI_BONG, CAU_THU, TRONG_TAI trên cả ba trạm nhằm giảm độ trễ đọc (Read latency)[1]. Lập luận này đúng một nửa, nhưng lại bỏ qua nửa còn lại tàn khốc hơn: Độ trễ ghi (Write latency).

Trong mùa giải, danh sách cầu thủ liên tục biến động (bổ sung cầu thủ thay thế do chấn thương, nhận thẻ phạt tích lũy). Nếu dữ liệu được nhân bản toàn phần trên các trạm phân tán, mọi thao tác UPDATE hoặc INSERT trên bảng CAU_THU tại trạm cục bộ sẽ phải đợi xác nhận từ tất cả các trạm khác để duy trì tính nhất quán đồng bộ (Synchronous Consistency) thông qua 2PC. Lượng băng thông và độ trễ tiêu tốn cho quá trình này sẽ triệt tiêu hoàn toàn lợi ích của việc đọc dữ liệu cục bộ. Thiết kế này thiếu vắng cơ chế giải quyết xung đột (Conflict Resolution) nếu hai trạm thực thi việc thay đổi trạng thái cầu thủ cùng một lúc.

6. Tổng Kết và Khuyến Nghị (Conclusions & Recommendations)

Báo cáo "Xây dựng hệ cơ sở dữ liệu quản lý một giải bóng đá chuyên nghiệp" là một tài liệu mang tính chất đối phó, sử dụng Trí tuệ Nhân tạo để che đậy sự yếu kém trong kiến thức hàn lâm. Văn bản khoác lên mình hình thức hoàn chỉnh, ngôn từ trôi chảy, nhưng bị bóc trần bởi các sai lầm toán học vô lý, sự lặp lại máy móc và kiến trúc hệ thống đầy rẫy các điểm mù.

Các đánh giá then chốt:

1. **Về tính Liêm chính Học thuật:** Việc sử dụng AI tạo sinh để làm giả các kết quả truy vấn và sinh ra một bảng xếp hạng với dữ liệu mâu thuẫn về tỷ lệ Thắng/Thua và tổng số trận là sự vi phạm trực tiếp các tiêu chuẩn báo cáo nghiên cứu. Hệ thống không hề tồn tại một quá trình nạp dữ liệu và kiểm thử thực tế.
2. **Về Phân tích Thiết kế (Data Modeling):** Báo cáo vi phạm các nguyên lý thiết kế cơ bản

khi lưu trữ dữ liệu dẫn xuất (bàn thắng/thẻ phạt) trong bảng giao dịch cốt lõi và thất bại hoàn toàn trong việc mô hình hóa dòng thời gian (Temporal database design) của các thực thể hợp đồng.

3. **Về Lý thuyết Chuẩn hóa:** Sinh viên không hiểu khái niệm cốt lõi của Dạng chuẩn Boyce-Codd (BCNF) về các khóa ứng viên gối nhau (overlapping candidate keys), dẫn đến việc đánh giá 11 lược đồ một cách hiển nhiên (trivial) và bỏ lọt các phụ thuộc hàm theo thời gian.
4. **Về Kỹ thuật Điều khiển Đồng thời:** Việc áp dụng lệnh SELECT COUNT(*) bên trong các Trigger mà không kết hợp với các hình thức khóa độc quyền (SELECT ... FOR UPDATE hoặc Advisory Locks)[10] mở ra lỗ hổng Race Condition nghiêm trọng, khiến dữ liệu bị hỏng trong các kịch bản tương tác đồng thời. Mã nguồn Stored Procedure cũng không tận dụng được cơ chế COMMIT/ROLLBACK[23].
5. **Về Kiến trúc Phân tán:** Việc áp dụng Schema nội bộ để mô phỏng một cơ sở dữ liệu phân tán vật lý cùng với việc đặt các thuộc tính "cứng" vào vị từ phân mảnh chứng tỏ lỗ hổng lớn về lý thuyết phân tán (Distributed Theory).

Dưới góc độ đánh giá của một Giảng viên phụ trách học phần chuyên sâu bậc Sau đại học về "Các Hệ thống Cơ sở dữ liệu", báo cáo này bị **từ chối nghiệm thu**. Yêu cầu nhóm sinh viên phải tái cấu trúc toàn diện ERD, bổ sung đầy đủ các giải thuật khóa đồng thời, áp dụng các kỹ thuật Kích thước thay đổi chậm (SCD) cho dữ liệu hợp đồng, và nghiêm cấm sử dụng các công cụ tạo sinh văn bản tự động để giải quyết các phần minh chứng tính toán. Kiến trúc của một hệ thống cơ sở dữ liệu không nằm ở việc cú pháp lệnh SELECT viết đẹp như thế nào, mà nằm ở khả năng hệ thống đó vẫn đứng vững dưới tác động của sự gia tăng đồng thời (concurrency) và tính phức tạp của dữ liệu.

Nguồn trích dẫn

1. BTL Các hệ thống CSDL.pdf
2. unknown_url
3. Boyce-Codd Normal Form (BCNF) - DevsUnite, https://devsunite.com/resources/core-subjects/dbms/boyce-codd-normal-form-bcnf
4. Difference between 3NF and BCNF in simple terms (must be able to, https://stackoverflow.com/questions/8437957/difference-between-3nf-and-bcnf-in-simple-terms-must-be-able-to-explain-to-an-8
5. PostgreSQL Timestamp: Data Types, Functions, and Best Practices, https://www.instaclustr.com/education/postgresql/postgresql-timestamp-data-types-functions-and-best-practices/
6. Documentation: 9.1: ALTER TABLE - PostgreSQL, https://www.postgresql.org/docs/9.1/sql-altertable.html
7. Documentation: 18: 5.12. Table Partitioning - PostgreSQL, https://www.postgresql.org/docs/current/ddl-partitioning.html
8. Database Normalization: 1NF vs 2NF vs 3NF Examples, https://sql-designer.com/blog/database-normalization
9. Normal Forms in DBMS: 1NF, 2NF, 3NF, BCNF, 4NF & 5NF,

https://www.simplilearn.com/normal-forms-in-dbms-article
10. Preventing Postgres SQL Race Conditions with SELECT FOR, https://on-systems.tech/blog/128-preventing-read-committed-sql-concurrency-errors/
11. Concurrency control in Postgres - Medium, https://medium.com/@devakrishna33/concurrency-control-in-postgres-7518503648ed
12. ACID, Read Phenomena, and Isolation Levels: What to Use When, https://abhimanyunagurkar.com/blog/acid-properties-and-isolation-levels/
13. Preventing Race Conditions with SELECT FOR UPDATE in Web, https://leapcell.io/blog/preventing-race-conditions-with-select-for-update-in-web-applications
14. Concurrency Anomalies Explained | VONNG, https://vonng.com/en/db/concurrent-control/
15. How to Handle Race Conditions in PostgreSQL Functions, https://oneuptime.com/blog/post/2026-01-25-postgresql-race-conditions/view
16. PostgreSQL trigger race condition updating a balance table from, https://stackoverflow.com/questions/75058984/postgresql-trigger-race-condition-updating-a-balance-table-from-transactions
17. How to deal with race conditions using Java and PostgreSQL, https://dev.to/ramoncunha/how-to-deal-with-race-conditions-using-java-and-postgresql-4jk6
18. Postgres Concurrency in Rails: Row Locks & Deadlocks (2026), https://danialasghar.com/blog/postgres-race-conditions-row-locks-rails/
19. Using PostgreSQL advisory locks to avoid race conditions, https://firehydrant.com/blog/using-advisory-locks-to-avoid-race-conditions-in-rails/
20. Postgres Counter Avoiding Race Conditions, https://dba.stackexchange.com/questions/319612/postgres-counter-avoiding-race-conditions
21. Inserting State Transitions in Postgres - Thoughtbot, https://thoughtbot.com/blog/inserting-state-transitions-in-postgres
22. PostgreSQL SELECT FOR UPDATE: Row-Level Locks Explained, https://www.cybertec-postgresql.com/en/select-for-update-considered-harmful-postgresql/
23. Oracle vs PostgreSQL - Transaction Control - Reply, https://www.reply.com/contents/Oracle_vs_Postgres_-_Transaction_Control_-_EN.pdf
24. stored procedures vs functions? : r/PostgreSQL - Reddit, https://www.reddit.com/r/PostgreSQL/comments/btqzn1/stored_procedures_vs_functions/
25. Function vs Stored Procedure in SQL - SQLPro Studio, https://www.sqlprostudio.com/blog/45-function-vs-stored-procedure-in-sql
26. PostgreSQL transaction management in procedures - Medium, https://franckpachot.medium.com/postgresql-transaction-management-in-proc

edures-9b57cb909067
27. Transitioning from Oracle to PostgreSQL: PL/SQL vs PL/pgSQL, https://stormatics.tech/blogs/transitioning-from-oracle-to-postgresql-pl-sql-vs-pl-pgsql
28. DAY-8 PSQL, Difference Function vs. Procedure - DEV Community, https://dev.to/elayaraj31/day-psql-difference-function-vs-procedure-28a5
29. PostGreSQL - Microsoft SQL Server - SQLPro - Developpez.com, https://sqlpro.developpez.com/tutorial/postgresal-vs-sqlserver/part3-very-extremely-detailed-comparison/
30. Avoid PostgreSQL Mutating Table: How to Prevent It - MinervaDB, https://minervadb.xyz/how-to-avoid-a-mutating-table-happening-in-postgresql/
31. ORA-04091 table is mutating trigger/function may not see it when, https://dba.stackexchange.com/questions/322759/ora-04091-table-is-mutating-trigger-function-may-not-see-it-when-cascading-work
32. Understanding Check Constraints in PostgreSQL - Severalnines, https://severalnines.com/blog/understanding-check-constraints-postgresql/
33. Subtract Seconds from a Time Value in PostgreSQL - Database.Guide, https://database.guide/subtract-seconds-from-a-time-value-in-postgresql/
34. 18: 9.9. Date/Time Functions and Operators - PostgreSQL, https://www.postgresql.org/docs/current/functions-datetime.html